CHAPTER D2

Claude models, prompting, and context engineering

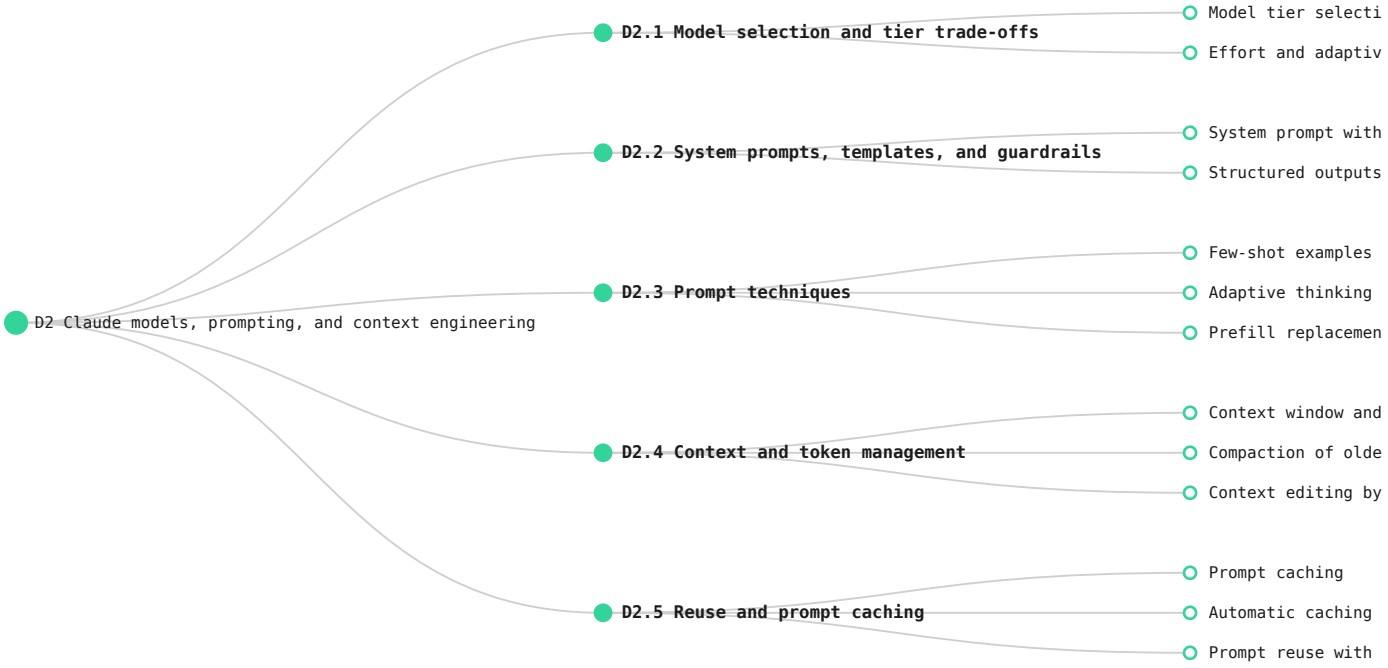
WEIGHT 13%. INFERENCE FROM SECONDARY SOURCES.

Lowest tier, tuned effort, counted tokens, cached prefix.

5 sections, 13 components, 11 recall cues.

CONCEPT MAP

Units branch to components. On a small screen, scroll sideways.



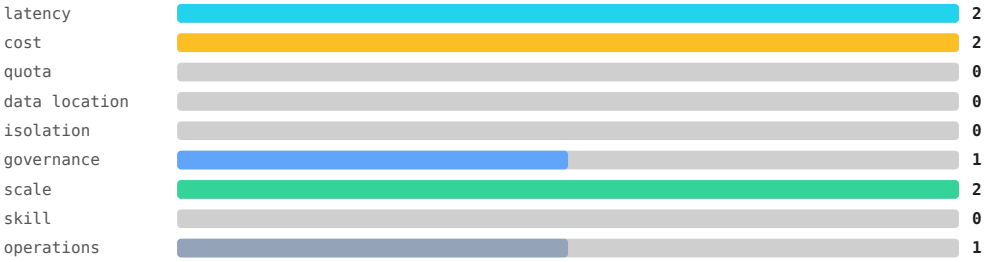
SECTIONS

D2.1 CHAPTER D2, CLAUDE MODELS, PROMPTING, AND CONTEXT ENGINEERING

Model selection and tier trade-offs

Lowest tier that clears the measured floor. Then tune effort.

CONSTRAINT FAMILIES



MODEL TIER SELECTION

Choose the lowest tier that clears a quality floor you measured on your own tasks.

Rule Pick the lowest tier that clears the measured quality floor, then tune effort on that tier.

Trap The top tier for every request looks safe, but it adds cost when evals show that a lower tier clears the floor.

Exam tip Name the measured floor in your reasoning before you name a tier.

EFFORT AND ADAPTIVE THINKING

Effort steers how much one model thinks, so tuning effort can come before a model change.

Rule Tune effort on the current model before you move to a larger tier, when the evals allow it.

Trap A move to the top tier for hard requests looks direct, but it skips effort, which is often a better lever than a model change.

Exam tip On Claude Haiku 5.5 with thinking disabled, a per-message effort change returns a 400 error, so use adaptive thinking to vary effort by turn.

▼ RECALL CUES (2)

1. **cut cost for a high-volume task with a measured quality floor**
   Answer lowest tier that clears the measured floor
   Rule Measure the floor first. Then choose the lowest tier that clears it.
   - Top tier for all traffic
   - A custom model before tier and effort tuning
2. **a subset of requests misses the quality floor on the lowest tier**
   Answer escalate the subset that misses the floor with effort or a higher tier
   Rule Escalate only the subset that misses the floor. Keep the rest on the lower tier.
   - Traffic sent to a higher tier by default
   - No quality check at all

▼ NUMBERS AND VERSIONS (13)

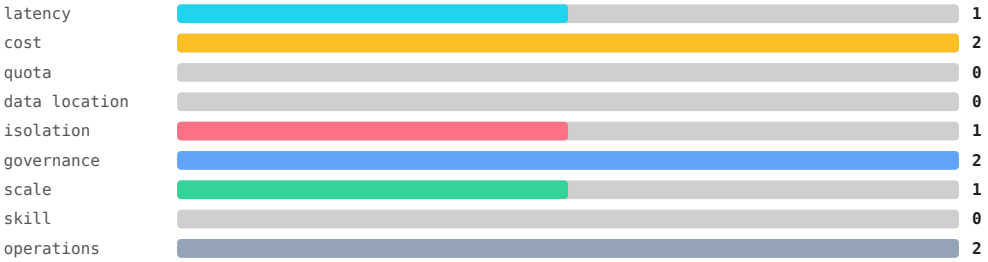
- Claude Haiku 5.5 costs $0.10 per million input tokens and $0.50 per million output tokens for prompts up to 100,000 tokens. source
- Claude Haiku 5.5 costs $0.50 per million input tokens and $2.50 per million output tokens for prompts over 100,000 tokens. source
- Claude Sonnet 5.5 costs $2 per million input tokens and $10 per million output tokens. source
- Claude Opus 5.5 costs $4 per million input tokens and $20 per million output tokens. source
- Claude Fable 5.1 costs $10 per million input tokens and $50 per million output tokens. source
- The four current models in the comparison table have a 1M token context window and 128K max output. source
- The model selection matrix says that most workloads start with Claude Opus 5.5. source
- The retirement date for Claude Haiku 5.5 is not sooner than October 7, 2027. source
- The default effort for Claude Haiku 5.5 is medium. source
- The default effort for Claude Opus 5.5 is medium. source
- Claude Haiku 5.5 supports the effort levels low, medium, high, xhigh, and max. source
- On Claude Haiku 5.5, a thinking budget_tokens setting returns a 400 error. source
- Per-message effort is in beta and needs the beta header mid-conversation-output-config-2026-07-01. source

D2.2 CHAPTER D2, CLAUDE MODELS, PROMPTING, AND CONTEXT ENGINEERING

System prompts, templates, and guardrails

Role and output rules in the prompt. Access rules in code.

CONSTRAINT FAMILIES



SYSTEM PROMPT WITH ROLE, SCOPE, AND RULES

The system prompt sets the role, scope, and rules, while your code enforces access (Inference).

Rule Put role and output rules in the system prompt. Put access rules in code.

Trap A rule in the system prompt looks like access control, but it is guidance that code must enforce (Inference).

Exam tip When a scenario needs a hard limit, choose code enforcement over a longer system prompt.

STRUCTURED OUTPUTS AS THE OUTPUT CONTRACT

Structured outputs constrain the response to a JSON schema, so parsing errors leave the code path.

Rule Use structured outputs when code parses the answer. Use prose when a person reads it.

Trap A prompt that asks for JSON only looks equal to a schema, but nothing constrains the text, so it can still break parsing.

Exam tip Check stop_reason for refusal or max_tokens before you parse, because both can return output that does not match the schema.

▼ RECALL CUES (2)

1. **user text must not change a rule**
   Answer code enforcement with guidance in the system prompt
   Rule Enforce access in code. Use the system prompt for guidance only.
   - A rule that appears only in the system prompt
   - A longer system prompt as a lock
2. **downstream code must parse the answer into fields**
   Answer structured outputs with a schema
   Rule Send the schema in output_config format. Check the stop reason before you parse.
   - A prompt that asks for JSON only
   - Pattern cleanup of prose

▼ NUMBERS AND VERSIONS (5)

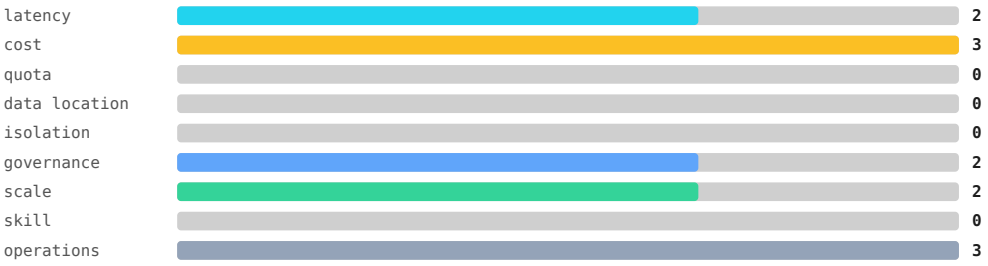
- A lightweight model such as Claude Haiku 5.5 can pre-screen user input before the main conversation. source
- A stop_reason refusal from the Claude Haiku 5.5 screen counts as a harmful verdict. source
- On Claude Haiku 5.5, Claude Sonnet 5.5, and Claude Opus 5.5, a role system message in messages keeps the system and message caches. source
- The explicit limits are 20 strict tools, 24 optional parameters, and 16 union-type parameters per request. source
- Compiled grammars stay cached for 24 hours from last use. source

D2.3 CHAPTER D2, CLAUDE MODELS, PROMPTING, AND CONTEXT ENGINEERING

Prompt techniques

Three to five diverse examples. Measure before adding more.

CONSTRAINT FAMILIES



FEW-SHOT EXAMPLES IN MULTISHOT PROMPTS

Examples show the output pattern, so Claude copies the format, tone, and structure they show.

Rule Add 3 to 5 diverse examples in example tags, then measure the result before you add more.

Trap More examples look like more guidance, but a set that lacks variety teaches an unintended pattern.

Exam tip When examples fail, check that they mirror the real input before you add more examples.

ADAPTIVE THINKING AND MANUAL CHAIN OF THOUGHT

Adaptive thinking gives the model room to reason, so manual step prompts are a fallback when thinking is off.

Rule Use adaptive thinking first. Use a manual step prompt only when thinking is off.

Trap A step-by-step plan in every prompt looks thorough, but adaptive thinking often reasons better without one.

Exam tip On Claude Haiku 5.5, a budget_tokens setting returns a 400 error, so use the effort level instead.

PREFILL REPLACEMENT

Claude 4.6 and later reject a prefilled final turn, so each prefill goal moves into the prompt or a schema.

Rule On Claude 4.6 and later, replace each prefill with instructions, structured outputs, or a tool.

Trap A repeat of the same prefill, unchanged, looks like a small fix, but the 400 error repeats on Claude 4.6 and later.

Exam tip A 400 error on a final assistant turn points to a prefill, so remove it before you change other settings.

▼ RECALL CUES (2)

1. **accuracy falls short of the bar on a classification task**
   Answer examples before a larger change
   Rule Add a few diverse examples, then run the evals before a larger change.
   - Custom model training first
   - Reasoning added to every call
2. **a prefill returns a 400 error on the newest models**
   Answer instructions, structured outputs, or a tool in place of the prefill
   Rule Move each prefill goal into the request. Do not send a prefilled final turn.
   - Retrying the same request
   - Thinking turned off as a fix

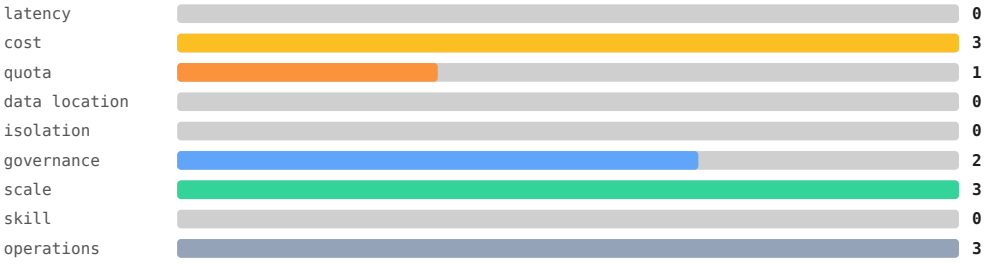
▼ NUMBERS AND VERSIONS (8)

- Include 3 to 5 examples for best results. source
- Queries at the end of a long prompt can improve response quality by up to 30 percent in tests. source
- Claude Haiku 5.5 uses adaptive thinking, and a budget_tokens setting returns a 400 error. source
- On Claude Haiku 5.5, thinking can be turned off only at high effort or below. source
- Forced tool use works with adaptive thinking. Claude Opus 5.5, Claude Sonnet 5.5, Claude Fable 5.1, and Claude Mythos 5.1 reject it with a 400 error. source
- The default effort for Claude Haiku 5.5 is medium. source
- Claude 4.6 and later models reject a prefilled final assistant turn with a 400 error. source
- Claude Opus 5.5, Claude Sonnet 5.5, Claude Fable 5.1, and Claude Mythos 5.1 reject forced tool use. Use tool_choice auto with strict tool use or structured outputs. source

Context and token management

Count with the model you call. Budget before you send.

CONSTRAINT FAMILIES



CONTEXT WINDOW AND TOKEN COUNTING

Every input and output token counts toward the context window, so count tokens before you send a turn.

Rule Count tokens with the model ID you will call, and budget each turn before you send it.

Trap A count from an older model looks reusable, but Claude 4.7 and later models use a newer tokenizer, so the count changes.

Exam tip A token count is an estimate, so keep a margin below the window limit. (Inference)

COMPACTION OF OLDER TURNS

Compaction replaces older turns with a summary that Claude writes on the server, so the active context stays small.

Rule Compact older turns when the task needs them, and keep recent turns word for word.

Trap A rolling window of the last turns looks cheap, but it drops early constraints that the task still needs (Inference).

Exam tip If a summary drops a constraint, write a custom summarization prompt before you change the model.

CONTEXT EDITING BY RULE

Context editing clears old tool results or thinking blocks by rule, so the context shrinks without a summary.

Rule Clear old tool results by rule. Keep thinking blocks when cache hits matter.

Trap A second summary call on every turn looks thorough, but a rule-based clear needs no extra call.

Exam tip If cache reads drop after a clear, check clear_at_least and the keep setting for thinking blocks.

▼ RECALL CUES (2)

1. **answer quality drops and cost rises as the history grows**
   Answer per-turn budget with compaction or a rule-based clear
   Rule Measure growth per turn. Then compact or clear the part that grows.
   - A bigger window as the fix
   - Full history on every turn
2. **a long agent task loses an early constraint after a summary**
   Answer custom summarization prompt with the constraint kept
   Rule Write a summarization prompt that keeps the constraint. Test it on later turns.
   - A summary on every turn
   - The default summary without a check

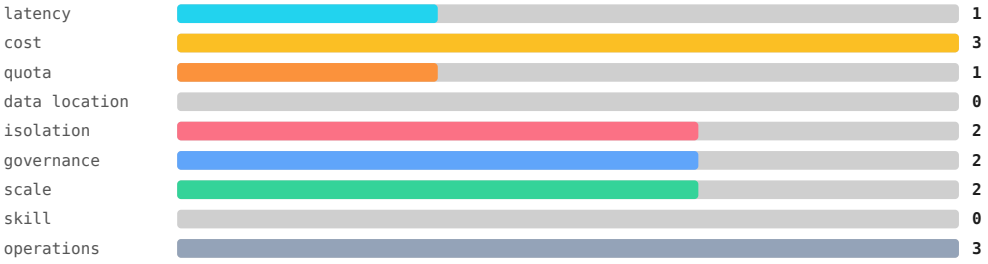
▼ NUMBERS AND VERSIONS (9)

- The context window for Claude Haiku 5.5 is 1M tokens. source
- Claude Haiku 5.5 prices input at $0.50 per million tokens for prompts over 100,000 tokens. source
- Claude Haiku 5.5 counts the same text as about 30 percent more tokens than Claude Haiku 4.5. source
- Compaction on demand uses the compact-2026-09-04 beta header. source
- Compaction at a token threshold uses the compact-2026-01-12 beta header. source
- Context editing is in beta and needs the context-management-2025-06-27 beta header. source
- The clear_tool_uses_20250919 strategy clears old tool results when context grows beyond a threshold. source
- The clear_thinking_20251015 strategy manages thinking blocks from earlier assistant turns. source
- On Claude Haiku 5.5, server-side context management never invalidates thinking blocks. source

Reuse and prompt caching

Breakpoint on the last block shared by every cached request.

CONSTRAINT FAMILIES



PROMPT CACHING

Pay full price for a stable prefix once, then read it at a fraction of the price while it stays identical.

Rule Put the breakpoint on the last block whose prefix is identical across the requests that must share the cache.

Trap A custom response cache in front of the API looks cheaper. But it adds a component, and it cannot reuse a prefix for a new question (Inference).

Exam tip If both cache token counts are zero, the prompt was below the minimum or the prefix changed.

AUTOMATIC CACHING VERSUS EXPLICIT BREAKPOINTS

Automatic caching places one breakpoint for you, and explicit breakpoints fix each cache point by hand.

Rule Use automatic caching for conversations that get longer and explicit breakpoints for sections that change at different rates.

Trap A top-level field on a request that already has 4 explicit breakpoints looks like extra protection, but the request fails with a 400 error.

Exam tip If a request returns a 400 error after a cache change, count the explicit breakpoints first.

PROMPT REUSE WITH TEMPLATES

A template fixes the stable instructions once and fills only the variable slots for each request.

Rule Put stable template text first, variable slots last, and version every change to the template.

Trap A rebuilt full prompt with a timestamp in each request looks harmless, but it breaks the cached prefix on every call.

Exam tip When a template change drops the hit rate to zero, check the block order before you change the TTL.

▼ RECALL CUES (3)

1. **reduce cost for a long prompt that repeats on every request**
   Answer prompt caching on the stable prefix
   Rule Cache what repeats. Move variable content after the breakpoint.
   - Smaller model that fails the quality floor
   - Custom response cache
2. **cache hit rate drops to zero after a prompt change**
   Answer prefix order or breakpoint position fix
   Rule Find the block that changes and move it after the breakpoint.
   - Longer TTL
   - More breakpoints on the same variable block
3. **a repeated instruction block and an output that a program must parse**
   Answer stable template with tagged slots and tagged output
   Rule Keep the template stable. Put variable content in its own tag after the stable text.
   - A rebuilt prompt with a timestamp
   - Parsing free text with a pattern match

▼ NUMBERS AND VERSIONS (18)

- The default cache lifetime is 5 minutes. source
- A 5-minute cache write costs 1.25 times the base input price. source
- A 1-hour cache write costs 2 times the base input price. source
- A cache read costs 0.1 times the base input price on Claude Haiku 5.5. It costs 0.05 times on Claude Opus 5.5 and Claude Sonnet 5.5, and 0.025 times on Claude Fable 5.1. source
- A request can define up to 4 cache breakpoints. source
- The lookback window checks at most 20 block positions per breakpoint. source
- The minimum cacheable prompt length is 512 tokens for Claude Fable 5.1, Claude Opus 5.5, Claude Sonnet 5.5, and Claude Haiku 5.5. source
- At 0.1 times the read price, the cache pays off after one read for a 5-minute write and after two reads for a 1-hour write. source
- Automatic caching uses a 5-minute TTL by default. source
- A ttl value of 1h sets a 1-hour TTL at 2 times the base input price. source
- Automatic caching uses one of the 4 available breakpoint slots. source
- In a request with mixed TTLs, 1-hour cache entries must appear before 5-minute cache entries. source
- Automatic caching is available on every platform except the legacy Amazon Bedrock integration for Claude Opus 4.6 and earlier. source
- An explicit cache_control on the last block with a different TTL makes automatic caching return a 400 error. source
- A cache hit requires 100 percent identical prompt segments up to the cache_control block. source
- Include 3 to 5 examples in examples tags for best results. source
- Queries placed at the end of a long prompt can improve response quality by up to 30 percent in tests. source
- Put long-form data near the top of the prompt for inputs of 20k or more tokens. source